UNIVERSITÀ DI PISA

DIPARTIMENTO DI INFORMATICA

Report
Assignment 4

Student:

Stefano Pea

ANNO ACCADEMICO 2024/2025

1 Introduction

The final assignment for the course focuses on parallelizing the Mergesort algorithm using FastFlow for shared-memory systems and MPI for distributed systems.

2 Execution Instructions

In order to test the programs, simply run the `sbatch_script_FF_CLUSTER.sh` script using the `sbatch` command inside the `spmcluster` machine.

```
sbatch sbatch_script_FF_CLUSTER.sh
```

This command will run `test_FF.sh`, that will start a battery of test for multiple number of threads and sizes of the array to be sorted. To test MPI, it is possible to call the script `MPI_tests_CLUSTER.sh` that will start a series of tests with 16 threads for the FastFlow intra-node sorting and various sizes for the array and for the number of nodes involved in the process. the results of the FF tests will be saved in the `output.txt`, while the MPI results will be saved in `mpi_output.txt` file. For the results and an analysis on them please consult Chapter 4.

3 Design and Implementation

Two programs have been developed for this assignment, one using only FastFlow and one combining FastFlow with MPI.

the `ffMS.cpp` file contains both the parallel FastFlow implementation and the sequential one, used for the speedup calculation and as a base case for the FF workers.

Some implementation choices have to be noted: in particular, we chose not to exchange the payload when communicating between nodes using MPI, but we simply exchange some sort of "dangling pointers". The tradeoff here is that the communication time decreases significantly but the only process that can access the real payloads is the master node.

After some considerations we concluded that it was a reasonable compromise and specifically for our problem, only the master node should access the payloads.

Another important consideration is that the number of MPI processes must be a power of 2. This is not a difficult requirement in our case and it greatly helps by simplifying the implementation. Similar consideration has to be done for the size of the initial Array(in the distributed version), which should be divisible by the number of nodes available. Here, the idea is similar as before, it greatly simplifies the calculation of how many elements to send to each node and can be easily solved by padding the array or with a more refined implementation.

In the FastFlow implementation, the farm is structured such that specifying `n` threads results in the creation of one master node and `n- 1` worker threads. When

`-t 1` is passed, our design choice is to spawn a single worker thread regardless, which leads to similar performance between the `-t 1` and `-t 2` test cases.

3.1 FastFlow

`ffMS.cpp` accepts various arguments:

- `-s N`: array size
- `-r R`: record payload in bytes
- `-t T`: number of FastFlow threads
- `-m M`: mode of execution (0 for sequential, 1 for fast flow parallelization)

The structured parallel pattern that we used for the parallelization is a farm of workers. Both the emitter and the collector have been merged inside a single node called "Master", that serve the role to distribute the work (tasks) among the other workers and pair runs to be merged.

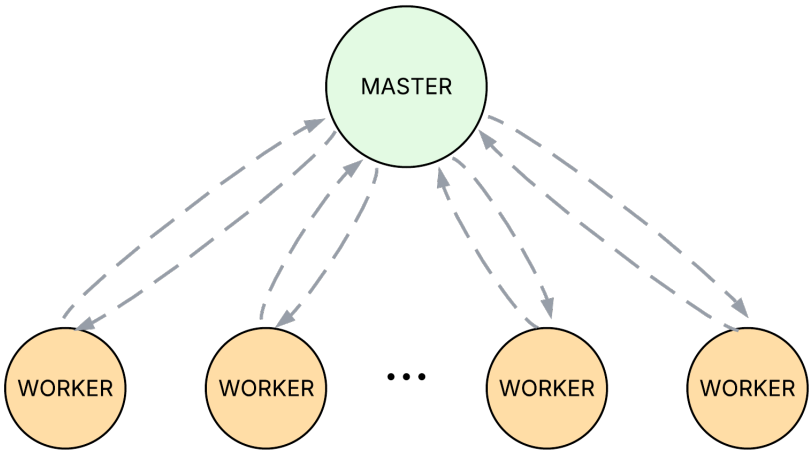


Figure 3.1: Structure of the FastFlow farm

The workflow of our farm is the following: First the Master node will split the original array in `number of threads - 1` unsorted chunks and will distribute them among the workers. Each of them will sort the received chunk using the sequential mergesort and will send back to the master the sorted run. At this stage the master node will check its queue to see if it has a run that can be merged with it and will create a task composed by the two sorted runs and will send it back to the workers for the merge. At the end the master node will have the original array sorted on the keys.

To determine the type of task received, each worker checks the task's level: if the level is 0, the task corresponds to an initial chunk that needs to be sorted; otherwise, it represents a merge operation between two already sorted runs.

We describe the algorithm as mergesort-like because it merges adjacent runs in the array, without strictly requiring them to be at the same hierarchical level as in traditional Mergesort. This relaxes the usual constraint of merging balanced pairs but still preserves correctness. The master node does not perform any sorting or merging; it only redistributes tasks to the workers. In practice, we do not observe a slowdown in execution times, likely because the merges are lightweight, and merging two adjacent runs early can be more efficient than waiting to satisfy all of Mergesort's balancing rules.

3.2 MPI

The MPI component was introduced to take advantage of the cluster infrastructure available to us. With access to 8 nodes, we extended the already parallelized FastFlow implementation to run across multiple nodes. In this setup, the node with ID 0 (referred to as the "Master" node) is responsible for scattering chunks of the input array to the other nodes. Each of these nodes then applies our existing FastFlow implementation to further divide and sort their assigned chunks using a local farm of worker threads.

After the "Worker" nodes end their sorting, a tree-like merge phase begins. Let us walk through this procedure assuming 4 nodes(1 "Master" and 3 "Worker" nodes):

Node 0 (Master) will send `Arraysize/4` elements to each node (Phase 1), and they will start sorting these elements with a call to a fastflow farm in the same way as we did in the shared-memory case (Phase 2).

They will then send back these sorted runs in a similar way to the one showed in Phase 3 and Phase 4.

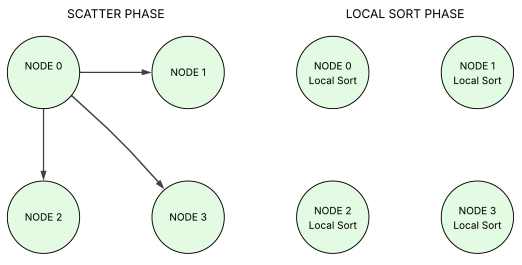


Figure 3.2: Phase 1 and 2

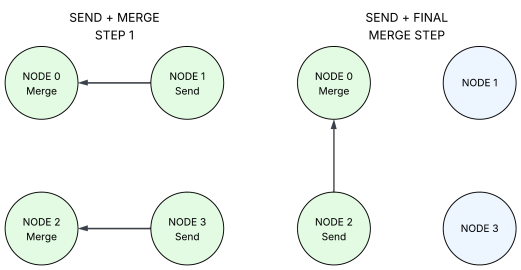


Figure 3.3: Phase 3 and 4

In our example, node 1 sends its sorted run to node 0, while node 3 sends its run to node 2. Nodes 0 and 2 then merge the received runs with their own local runs. In the following phase, node 2 sends its merged run to node 0, which performs the final merge step. As a result, node 0 ends up holding the fully sorted array.

It's important to note that the previous description simplifies the actual behavior. In practice, nodes 0 and 2 call non-blocking receive operations before starting to sort their local chunks or perform merges. This allows computation to proceed while waiting for incoming data, effectively overlapping communication and computation to improve overall performance.

4 Tests & Performance Analysis

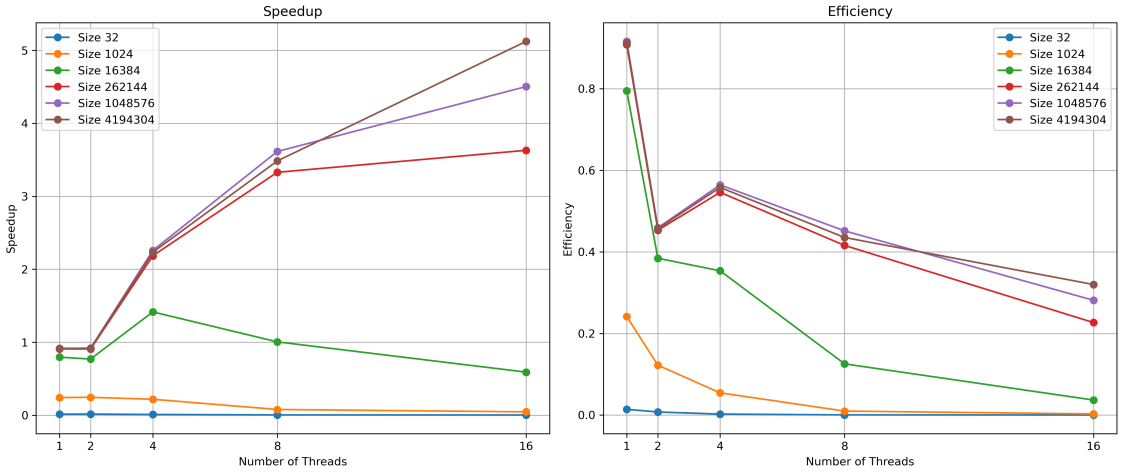
Several tests were conducted to evaluate the effectiveness of our implementations.

In particular, for the FastFlow component, we measured speedup and efficiency relative to the sequential mergesort implementation. We tested different array sizes and number of FF worker threads to observe how the system scales.

The size of the payload was not considered in the analysis, for the reason found in Section 3: we exchange dangling pointers, which remain unchanged regardless of the payload size.

For the MPI component we tested weak and strong scalability varying the number of nodes and the number of elements to be sorted. the number of FF threads in each node has been set to the number of threads that achieved the best speedup (16 in our case).

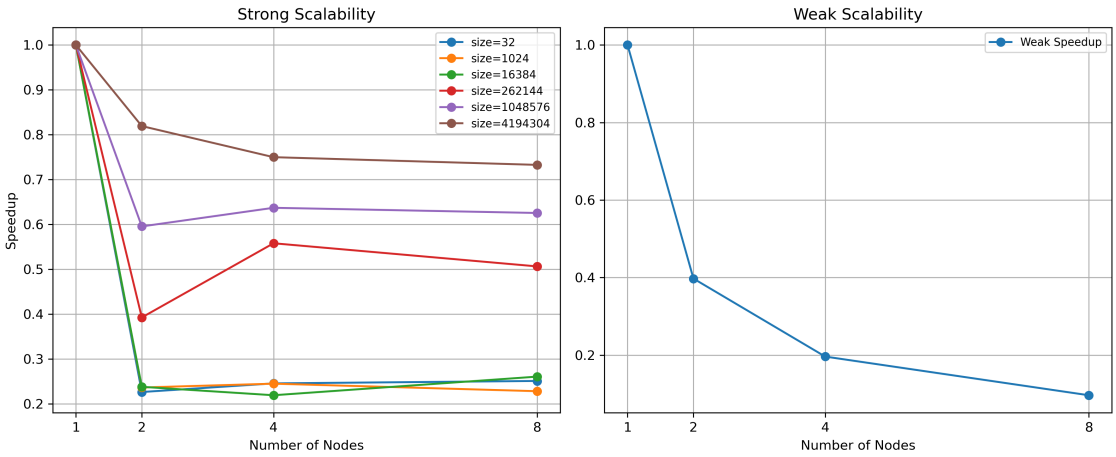
4.1 FastFlow



Regarding FastFlow, the battery of tests showed some pretty interesting results: first of all we see that we were able to achieve a speedup of a bit more than 5 with 16 thread and a size of 2^{22}, which is not great but considering the light computational nature of the problem, is, in our opinion, a decent result. The graph also shows that for very small sizes, the overhead introduced by FastFlow is detrimental to the effectiveness of our program. For small inputs, spawning workers and distributing

tasks between them takes more time than just running the sequential algorithm on the total array.

4.2 MPI



The situation when passing from shared-memory to distributed systems differs greatly.

For small array sizes, using more than one node offers no performance benefit. This is primarily due to the dominance of MPI setup and communication overhead relative to the minimal cost of sorting a small array locally.

For larger input sizes, performance improves; however, increasing the number of nodes also increases inter-node communication, which can negatively impact the overall execution time. This communication overhead becomes a limiting factor.

This effect is particularly evident in the weak scalability results: even when each node receives the same computational load, overall scalability decreases as the number of nodes increases, thus indicating that communication costs grow with the nodes.

5 Future Work

Some improvement can be made to our programs: first of all, we could implement a more strict merge steps in the FF implementation, merging runs exactly as the sequential algorithm.

Then we could remove the MPI limitations regarding the number of nodes and the size of the array to be sorted, but, as stated before, the complexity of our code will increase.

It is also possible, but more tests are necessary, that reducing the number of communications inter-node, maybe having the master node gather all the sorted

runs and then merging them locally, could lead to some form of speedup.